



Pengembangan Mekanika Simulasi Lingkungan Dinamis Menggunakan Grid World Matrix dan FSM pada Game 2D

Defanda Yeremia¹, Galih Adhi Kusuma², Firschanya Alula²

¹ Fasilkom, Teknik dInformatika, Universitas Esa Unggul, Jakarta, Indonesia

² Fasilkom, Teknik Informatika, Universitas Esa Unggul, Jakarta, Indonesia

Abstrak— Perkembangan game simulasi ekologi saat ini masih sering menyederhanakan mekanika lingkungan dengan logika yang tidak terstruktur, sehingga memicu *bottleneck* performa dan menurunkan tingkat responsivitas permainan. Penelitian ini bertujuan untuk merancang, mengimplementasikan, serta menganalisis performa arsitektur mekanika simulasi lingkungan dinamis 2D berbasis Unity Engine. Solusi yang ditawarkan adalah penggabungan struktur data Grid World Matrix untuk pemetaan spasial efisien dengan kompleksitas $O(1)$ dan algoritma Finite State Machine (FSM) untuk mengelola transisi status ekologis selular secara deterministik. Metodologi pengembangan sistem mengacu pada kerangka Game Development Life Cycle (GDLC). Pengujian dilakukan melalui pengujian Black-Box, *benchmark* performa Unity Profiler, evaluasi System Usability Scale (SUS), dan User Acceptance Testing (UAT). Hasil penelitian menunjukkan pengujian Black-Box mencapai tingkat keberhasilan 100% pada seluruh skenario uji. Pada skala grid standar 50×50 (2.500 sel), sistem mampu menjaga stabilitas performa dengan rata-rata 136,5 FPS, alokasi RAM 64,3 MB, dan waktu eksekusi FSM *tick* sebesar 0,48 milidetik. Selain itu, evaluasi tingkat kebergunaan menghasilkan skor SUS sebesar 82,00 yang masuk dalam kategori Grade A (Excellent), serta skor UAT sebesar 89,7% dengan kriteria Sangat Memuaskan. Dengan demikian, kombinasi Grid World Matrix dan FSM terbukti efektif dan efisien dalam menghasilkan simulasi lingkungan yang responsif dan terstruktur.

Kata Kunci: Finite State Machine; Game 2D; Grid World Matrix; Simulasi Lingkungan; Unity Engine

Abstract— The development of ecology simulation games currently still simplifies environmental mechanics using unstructured logic, causing performance bottlenecks and reducing game responsiveness. This study aims to design, implement, and analyze the performance of a 2D dynamic environment simulation mechanics architecture based on the Unity Engine. The proposed solution combines the Grid World Matrix data structure for efficient spatial mapping with $O(1)$ complexity and the Finite State Machine (FSM) algorithm to deterministically manage cellular ecological state transitions. The system development methodology follows the Game Development Life Cycle (GDLC) framework. Testing was conducted using Black-Box testing, Unity Profiler performance benchmarking, System Usability Scale (SUS) evaluation, and User Acceptance Testing (UAT). The results showed that Black-Box testing achieved a 100% success rate across all test scenarios. On a standard 50×50 grid scale (2,500 cells), the system maintained performance stability with an average of 136.5 FPS, 64.3 MB RAM allocation, and an FSM tick execution time of 0.48 milliseconds. Furthermore, usability evaluation yielded an SUS score of 82.00, categorized as Grade A (Excellent), and a UAT score of 89.7%, categorized as Very Satisfactory. Thus, combining Grid World Matrix and FSM proves effective and efficient in creating responsive and structured environmental simulations.

Keywords: 2D Game; Environmental Simulation; Finite State Machine; Grid World Matrix; Unity Engine

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri media interaktif, khususnya *video game*, telah mengalami pergeseran paradigma yang signifikan dalam beberapa dekade terakhir. *Game* tidak lagi dipandang semata-mata sebagai sarana hiburan komersial (*entertainment*), melainkan telah berkembang menjadi medium kompleks yang mampu menyampaikan pesan edukatif, simulasi sistem nyata, serta media kesadaran sosial dan lingkungan (*serious games*). Salah satu isu global yang krusial dan semakin sering diangkat ke dalam ranah media interaktif adalah degradasi lingkungan, perubahan iklim, dan krisis ekosistem pasca-industri. Konsep ini sejalan dengan tujuan pembangunan berkelanjutan (*Sustainable Development Goals / SDGs*) poin ke-15, yaitu *Life on Land*, yang menekankan pentingnya perlindungan, pemulihan, dan peningkatan penggunaan ekosistem daratan secara berkelanjutan.

Dalam konteks perancangan *game* simulasi ekologi atau *survival-restoration*, tantangan utama yang dihadapi oleh pengembang adalah bagaimana menyajikan simulasi lingkungan yang tidak hanya dinamis dan responsif, tetapi juga realistis, terprediksi, serta efisien secara komputasi. Lingkungan yang dinamis mengandaikan bahwa setiap tindakan atau intervensi yang dilakukan oleh pemain—seperti membersihkan polusi, menanam vegetasi, atau menyiram tanah—akan memberikan dampak langsung dan sistematis terhadap kondisi dunia *game*. Tanpa mekanisme simulasi yang responsif, interaksi pemain akan terasa dangkal dan tidak mampu memberikan pengalaman bermain yang mendalam (*immersive*) maupun menyampaikan pesan edukatif secara efektif.

Untuk mewujudkan simulasi lingkungan 2D yang kaya akan interaksi, dibutuhkan fondasi representasi ruang dan logika pemrosesan status yang kuat. Representasi ruang dalam *game* 2D berperspektif *top-down* umumnya mengandalkan struktur data berbasis kisi (*grid*). Pendekatan *Grid World Matrix* menawarkan solusi elegan untuk membagi dunia *game* menjadi elemen-elemen selular (*tile*) yang terintegrasi secara matematis. Setiap koordinat matriks mewakili area spesifik



di dalam peta yang menyimpan berbagai atribut ekologis, seperti tingkat kesuburan, kelembapan, tingkat pencemaran, hingga jenis vegetasi yang tumbuh di atasnya. Struktur ini memungkinkan pengembang melakukan manipulasi spasial secara konsisten dan efisien.

Namun demikian, struktur data matriks saja tidak cukup untuk mengelola perubahan perilaku dan transisi status lingkungan dari waktu ke waktu. Di sinilah peran algoritma *Finite State Machine* (FSM) menjadi sangat krusial. FSM merupakan model komputasi matematis yang mengorientasikan sistem ke dalam sejumlah status (*states*) terhitung, di mana transisi antarstatus dipicu oleh kondisi atau input tertentu. Ketika diterapkan pada setiap sel dalam *Grid World Matrix*, FSM memungkinkan setiap unit lingkungan bertindak sebagai mesin status mandiri. Sebagai contoh, sebuah sel tanah dapat bertransisi dari status *Barren* (Tandus/Tercemar), menjadi *Restoring* (Dalam Pemulihan), *Fertile* (Subur), hingga akhirnya *Flourishing* (Ekosistem Stabil) berdasarkan kriteria lingkungan dan interaksi pemain.

1.2 Identifikasi Masalah dan Kebaruan

Sebagian besar *game* simulasi 2D konvensional di pasaran kerap kali menyederhanakan mekanika lingkungan. Perubahan status objek lingkungan sering kali bersifat biner (misalnya: ada/tidak ada, tumbuh/mati) atau diimplementasikan menggunakan logika pemrograman yang tidak terstruktur secara terpusat (*spaghetti logic*). Hal ini berpotensi menimbulkan *bottleneck* performa ketika jumlah *tile* yang harus diproses meningkat secara eksplisif, serta menyulitkan proses *debugging* dan pengisian konten baru (*scalability*).

Penelitian-penelitian sebelumnya telah mengeksplorasi penggunaan FSM untuk pengatur perilaku karakter non-pemain (NPC) atau AI musuh. Namun, penerapan kombinasi *Grid World Matrix* dan FSM secara khusus untuk mensimulasikan dinamika ekosistem lingkungan 2D secara granular masih relatif terbatas. Integrasi kedua metode ini menghadirkan kebaruan (*state-of-the-art*) berupa arsitektur mekanika simulasi ekologi modular berbasis sel. Pendekatan ini memungkinkan terciptanya perilaku ekosistem yang tampak kompleks dan dinamis (*emergent behavior*) melalui aturan-aturan lokal FSM yang terdefinisi dengan jelas pada setiap sel matriks, tanpa membebani pemrosesan komputasi secara berlebihan pada *engine game* seperti Unity.

1.3 Urgensi dan Tujuan Penelitian

Urgensi dari penelitian ini terletak pada kebutuhan akan arsitektur rekayasa perangkat lunak *game* yang efisien, terstruktur, dan terukur dalam membangun gim simulasi ekologi. Dengan memanfaatkan kombinasi *Grid World Matrix* dan FSM, pengembang dapat menciptakan mekanika simulasi lingkungan dinamis yang stabil, responsif, serta mudah dipelihara. Hal ini menjadi relevan tidak hanya bagi pengembangan gim komersial berskala indie, tetapi juga untuk pengembangan media edukasi interaktif berbasis gamifikasi restorasi lingkungan.

Berdasarkan latar belakang dan identifikasi masalah di atas, tujuan utama dari penelitian ini adalah:

1. Merancang dan mengimplementasikan arsitektur data *Grid World Matrix* untuk pemetaan spasial dan penyimpanan atribut lingkungan pada *game* 2D berbasis Unity Engine.
2. Mengintegrasikan algoritma *Finite State Machine* (FSM) pada setiap sel matriks untuk mengelola transisi status ekologis dan respon terhadap intervensi pemain secara dinamis.
3. Menganalisis performa dan efektivitas kombinasi *Grid World Matrix* dan FSM dalam mendukung mekanika *gameplay* simulasi restorasi lingkungan yang realistis dan responsif.

2. METODOLOGI PENELITIAN

2.1 Alur Pengembangan Sistem (*Game Development Life Cycle* - GDLC)

Dalam penelitian ini, metodologi pengembangan yang digunakan adalah *Game Development Life Cycle* (GDLC). GDLC dipilih karena menyediakan kerangka kerja terstruktur yang dirancang khusus untuk mengakomodasi fleksibilitas, iterasi artistik, serta kompleksitas teknis dalam pembuatan media interaktif. Tahapan GDLC dalam penelitian ini dibagi menjadi lima fase utama, yaitu:

1. **Pra-produksi (*Pre-Production*):** Tahap awal yang berfokus pada penyusunan konsep dasar *game*, pembuatan Dokumen Desain Game (*Game Design Document* - GDD), penentuan gaya visual 2D, serta perancangan arsitektur sistem. Pada fase ini, pemodelan matematis untuk *Grid World Matrix* dan diagram alur *Finite State Machine* (FSM) dirancang secara terperinci.
2. **Produksi (*Production*):** Fase eksekusi teknis yang meliputi pembuatan aset grafis 2D, pengkodean sistem berbasis C# pada Unity Engine, integrasi arsitektur *Grid World Matrix* untuk pemetaan spasial, serta penerapan logika FSM pada setiap sel lingkungan.



3. **Pengujian (Testing):** Tahap pengujian fungsi sistem secara menyeluruh, mencakup validasi transisi status FSM, pengujian efisiensi memori dan *frame rate* (FPS), serta uji kelayakan pengguna.
4. **Pelepasan (Release):** Tahap kompilasi versi final *game* (*executable build*) yang siap untuk didistribusikan kepada pengguna atau responden pengujian.
5. **Pasca-produksi (Post-Production):** Tahap evaluasi hasil pengujian, pengumpulan umpan balik pengguna, dokumentasi kode, serta identifikasi area perbaikan untuk pengembangan di masa depan.

2.2 Perancangan Struktur Data *Grid World Matrix*

Grid World Matrix dirancang sebagai fondasi pemetaan ruang dua dimensi (2D) dalam *game*. Dunia *game* dibagi menjadi kisi-kisi selular berskala seragam, di mana setiap sel diwakili oleh indeks spasial dua dimensi (X, Y) dalam struktur data matriks. Pendekatan matriks ini memungkinkan pencarian (*lookup*) data lokasi secara konstan dengan kompleksitas waktu $O(1)$.

Setiap sel matriks merepresentasikan satu unit lahan ekologis dan menyimpan serangkaian variabel status (*state variables*) serta atribut dinamis, antara lain:

1. **Status Kebersihan/Polusi (Pollution Level):** Menandakan tingkat pencemaran lingkungan akibat limbah.
2. **Tingkat Kesuburan Tanah (Fertility Level):** Mengukur kemampuan sel untuk menumbuhkan vegetasi.
3. **Tingkat Kelembapan (Moisture Level):** Dipengaruhi oleh interaksi penyiraman atau kondisi cuaca.
4. **Jenis Vegetasi (Vegetation Type):** Menentukan jenis tanaman yang menduduki sel matriks tersebut.
5. **Status FSM Aktif (Current FSM State):** Mengidentifikasi kondisi ekologis sel pada detik pemrosesan tertentu.
6. Konversi antara koordinat dunia nyata (*World Position*) Unity ke dalam indeks matriks dilakukan melalui transformasi linier sederhana berdasarkan ukuran sel (*Cell Size*) dan titik acuan (*Origin Vector*). Mekanisme ini memastikan bahwa setiap interaksi klik atau posisi karakter pemain dapat diterjemahkan secara presisi ke dalam sel matriks yang tepat.

2.3 Pemodelan *Finite State Machine* (FSM) untuk Ekosistem Selular

Untuk mengelola dinamika perubahan lingkungan secara transparan dan efisien, setiap sel dalam *Grid World Matrix* diintegrasikan dengan mesin status terhingga atau *Finite State Machine* (FSM). Pemodelan FSM terdiri dari empat komponen utama: himpunan status (S), himpunan input/kejadian (I), fungsi transisi (δ), dan status awal (S_0). Status ekologis sel didefinisikan ke dalam empat kondisi utama:

1. **Status Barren (Tandus/Tercemar):** Status awal di mana sel tertutup polusi dan tidak dapat ditanami vegetasi.
2. **Status Restoring (Dalam Pemulihan):** Status transisi yang dicapai setelah pemain melakukan tindakan pembersihan limbah pada sel tersebut.
3. **Status Fertile (Subur):** Status di mana sel telah bersih dari polusi dan memiliki kelembapan serta kesuburan yang cukup untuk ditanami.
4. **Status Flourishing (Ekosistem Stabil):** Status optimal di mana vegetasi tumbuh sempurna, memberikan kontribusi positif terhadap skor pemulihan ekosistem secara keseluruhan.

Transisi antarstatus dikendalikan oleh fungsi $\delta(S, I) \rightarrow S'$. Input (I) dapat berasal dari dua sumber: interaksi langsung pemain (seperti tindakan membersihkan polusi, menyiram, atau menanam) dan pembaruan berkala sistem (*environment tick update*). Penggunaan FSM memastikan bahwa transisi status sel berjalan secara deterministik dan terisolasi dari sel lain, mencegah terjadinya ketidakstabilan data.

2.4 Metode Pengujian dan Evaluasi

Pengujian sistem dilakukan secara komprehensif melalui tiga pendekatan utama untuk memastikan kualitas teknis dan pengalaman pengguna:

1. **Pengujian Fungsionalitas (Black-Box Testing):** Berfokus pada verifikasi logika transisi status FSM pada *Grid World Matrix*. Pengujian ini bertujuan untuk memastikan seluruh kombinasi input interaksi pemain dan pembaruan sistem menghasilkan transisi status yang sesuai dengan spesifikasi perancangan.
2. **Evaluasi Usability (System Usability Scale - SUS):** Mengukur tingkat kebergunaan dan kemudahan antarmuka serta mekanika *game* bagi pengguna. Pengujian SUS menggunakan kuesioner standar yang terdiri dari 10 item



pertanyaan dengan skala Likert 1–5, di mana skor akhir dihitung untuk menentukan kategori kelayakan (*Acceptability Range*).

3. **Pengujian Penerimaan Pengguna (*User Acceptance Testing* - UAT):** Dilakukan terhadap sekelompok responden untuk mengukur tingkat kepuasan pengguna terhadap mekanika simulasi lingkungan, kejelasan informasi visual, dan daya tarik *gameplay* restorasi ekosistem

3. HASIL DAN PEMBAHASAN

3.1 Implementasi Sistem dan Arsitektur Metode

3.1.1 Struktur Data Grid World Matrix dalam Unity C#

Pada penelitian ini, *Grid World Matrix* diimplementasikan sebagai struktur data dua dimensi utama yang mengelola representasi spasial dan status atribut ekologi dalam lingkungan game 2D berbasis Unity Engine. Dunia game direpresentasikan sebagai matriks terstruktur berukuran $N \times M$ sel, di mana setiap posisi sel menampung *instance* dari objek kelas Cell. Struktur ini dirancang secara modular menggunakan bahasa pemrograman C# untuk menjamin efisiensi alokasi memori dan kecepatan akses data spasial dengan kompleksitas waktu $O(1)$.

Secara teknis, pengorganisasian matriks dikendalikan oleh kelas *manager* terpusat bernama GridManager. Kelas ini bertanggung jawab melakukan inisialisasi kisi, mengelola pemetaan koordinat, serta menyediakan *Application Programming Interface* (API) untuk interaksi komponen lain seperti karakter pemain atau sistem cuaca. Proses konversi antara posisi koordinat dunia nyata dalam Unity (*World Position*) dan indeks matriks (*Grid Index*) dilaksanakan melalui persamaan matematis linier berikut:

$$X_{\text{grid}} = \left\lfloor \frac{X_{\text{world}} - X_{\text{origin}}}{\text{CellSize}} \right\rfloor$$

$$Y_{\text{grid}} = \left\lfloor \frac{Y_{\text{world}} - Y_{\text{origin}}}{\text{CellSize}} \right\rfloor$$

Sebaliknya, untuk menentukan posisi spasial pusat sel di dunia game berdasarkan indeks matriks, digunakan formula pemetaan balik:

$$X_{\text{world}} = X_{\text{origin}} + (X_{\text{grid}} \times \text{CellSize}) + \left(\frac{\text{CellSize}}{2} \right)$$

$$Y_{\text{world}} = Y_{\text{origin}} + (Y_{\text{grid}} \times \text{CellSize}) + \left(\frac{\text{CellSize}}{2} \right)$$

Melalui arsitektur ini, setiap sel pada gridArray beroperasi secara independen tanpa memerlukan komponen MonoBehaviour terpisah untuk masing-masing sel. Pendekatan ini secara drastis mengurangi beban *overhead* memori pada Unity Engine, karena pengikatan komponen *game object* (seperti *Tilemap Renderer*) dilakukan secara terpusat melalui manajemen kejadian (*event-driven rendering*).

3.1.2 Pemodelan dan Implementasi Finite State Machine (FSM) Selular

Untuk mengendalikan dinamika perubahan lingkungan secara deterministik dan terstruktur, setiap unit sel pada Grid World Matrix diprogram mengacu pada arsitektur *Finite State Machine* (FSM). FSM selular ini mengatur transisi antara empat status lingkungan yang valid: Barren, Restoring, Fertile, dan Flourishing.

Pengaturan kondisi transisi FSM didasarkan pada ambang batas (*threshold*) variabel atribut ekologis serta *trigger* kejadian yang dihasilkan dari aksi pemain (*player action*) maupun pembaruan periodik sistem (*environment tick*). **Tabel 1** memperlihatkan matriks transisi status FSM yang mengatur perilaku dinamis setiap sel lingkungan.

Tabel 1. Matriks Transisi Status Finite State Machine (FSM) Lingkungan



Status Awal	Kejadian / Trigger Input	Syarat Kondisi Atribut	Status Akhir	Visual Rendering
Barren	Player: Cleaning Action	$PollutionLevel \leq 0.3f$	Restoring	Tanah Pemulihan (Cokelat Muda)
Restoring	Player: Watering / Fertile Tick	$Moisture \geq 0.5f \wedge Pollution \leq 0.05f$	Fertile	Tanah Subur (Cokelat Gelap)
Fertile	Player: Planting Seed	$PlantType \neq "None" \wedge Fertility \geq 0.6f$	Flourishing	Vegetasi Tumbuh (Hijau Rindang)
Flourishing	Environment Tick: Neglect / Pollution	$Pollution > 0.4f \vee Moisture < 0.1f$	Barren	Tanah Tandus (Abu-abu / Sampah)
Fertile	Environment Tick: Drought Event	$Moisture < 0.15f$	Restoring	Tanah Pemulihan Kering

FSM selular diimplementasikan menggunakan pola eksekusi terstruktur pada metode `ProcessStateTransition()`.

3.1.3 Mekanika Gameplay dan Visualisasi Restorasi Lingkungan

Penerapan kombinasi Grid World Matrix dan FSM menghasilkan mekanisme interaksi *gameplay* yang responsif dan langsung. Ketika pemain mengarahkan kursor dan melakukan aksi interaksi (misalnya menggunakan alat pembersih polusi), sistem melakukan kalkulasi koordinat spasial untuk menemukan sel terpengaruh pada `gridArray`.

Apabila sel berada pada status Barren, aksi pembersihan mengurangi nilai `pollutionLevel`. Begitu nilai polusi mencapai ambang batas di bawah $0.3f$, FSM secara otomatis mengubah status sel menjadi Restoring. Pembaruan status ini memicu *event* `OnStateChanged`, yang secara instan memperbarui ubin grafis pada Unity Tilemap tanpa perlu merender ulang keseluruhan peta. Siklus ini berlanjut ketika pemain menyiram tanah dan menanam benih, secara bertahap merubah lanskap lingkungan dari wilayah pasca-apokaliptik yang tercemar menjadi vegetasi hijau yang subur.

3.2 Hasil Pengujian Fungsionalitas dan Logika FSM (Black-Box Testing)

3.2.1 Skenario Uji Transisi Status Sel

Untuk memastikan keterandalan dan kebenaran logika FSM pada seluruh sel Grid World Matrix, dilakukan pengujian fungsionalitas menggunakan metode *Black-Box Testing*. Pengujian ini berfokus pada verifikasi bahwa setiap kombinasi *input* interaksi pemain dan kondisi pembaruan lingkungan menghasilkan transisi status yang akurat sesuai dengan matriks perancangan.



Tabel 2 menyajikan rangkuman hasil pengujian fungsionalitas logika FSM pada sistem simulasi lingkungan yang mencakup 10 skenario uji utama.

Tabel 2. Hasil Pengujian Black-Box Logika Transisi FSM Selular

ID	Skenario Pengujian	Input / Kondisi Awal	Ekspektasi Output Status	Hasil Aktual	Status
TS01	Pembersihan Polusi Sel Barren	Barren (Pollution 1.0 → 0.2)	Restoring	Restoring	PASS
TS02	Pembersihan Parsial Polusi	Barren (Pollution 1.0 → 0.5)	Tetap Barren	Barren	PASS
TS03	Penyiraman Air Sel Restoring	Restoring (Moisture 0.1 → 0.6)	Fertile (Fertility ≥ 0.5)	Fertile	PASS
TS04	Penanaman Benih Sel Fertile	Fertile (PlantType = "Oak")	Flourishing	Flourishing	PASS
TS05	Penanaman Benih Sel Barren	Barren (PlantType = "Oak")	Aksi Ditolak (Tetap Barren)	Barren	PASS
TS06	Pencemaran Sel Flourishing	Flourishing (Pollution = 0.6)	Barren (Tanaman Mati)	Barren	PASS
TS07	Kekeringan Ekstrem Sel Fertile	Fertile (Moisture = 0.05)	Restoring	Restoring	PASS
TS08	Klik di Luar Batas Matriks	WorldPos (−100, −100)	Menolak Aksi (Return Null)	Return Null	PASS
TS09	Pembaruan Massal 2500 Sel	Sel Bervariasi (Batch Tick)	Transisi Paralel Sesuai FSM	Berjalan Normal	PASS
TS10	Pemulihan Berantai Tetangga	Pollution Spreading Event	Status Menyesuaikan Polusi	Sesuai Logika	PASS



Berdasarkan hasil pengujian pada Tabel 2, seluruh 10 skenario pengujian fungsionalitas dinyatakan lulus (**PASS**). Sistem berhasil merespons *input* interaksi pemain dengan tepat dan menolak interaksi yang tidak valid, seperti mencoba menanam benih pada sel yang masih dalam kondisi Barren atau tercemar.

3.2.2 Validasi Kestabilan Sistem dan Edge Cases

Selain uji skenario normal, dilakukan pengujian terhadap kondisi batas (*edge cases*). Salah satu skenario krusial adalah penanganan batas indeks matriks (*array index out of bounds*). Ketika pemain mengeklik area di luar koordinat Grid World Matrix, fungsi `GetCellFromWorldPosition()` mengembalikan nilai null dan sistem penanganan *error* secara efektif menghentikan eksekusi tanpa menyebabkan *crash* pada aplikasi.

Selain itu, pengujian beban transisi beruntun (*cascading state updates*) saat 2.500 sel mengalami pembaruan status secara bersamaan dalam satu *environment tick* membuktikan bahwa FSM mampu mengeksekusi transisi status secara konsisten tanpa ada data sel yang korup atau terlewat.

3.3 Pengujian Performa Komputasi dan Efisiensi Memori

3.3.1 Benchmark Frame Rate (FPS) dan Penggunaan Memori

Tantangan utama pada game simulasi berbasis kisi adalah efisiensi komputasi ketika ukuran matriks diperbesar. Untuk mengukur batas skalabilitas metode Grid World Matrix dan FSM, dilakukan pengujian *benchmark* performa menggunakan Unity Profiler pada perangkat uji berkecepatan prosesor Intel Core i7-11800H, RAM 16GB, dan GPU Nvidia RTX 3060.

Pengujian dilakukan dengan mengukur dua parameter kinerja utama: *Frame Rate* rata-rata (*Frames Per Second* / FPS) dan Alokasi Memori Utama (RAM). Variasi ukuran matriks yang diuji meliputi 10×10 (100 sel), 25×25 (625 sel), 50×50 (2.500 sel), 100×100 (10.000 sel), hingga 200×200 (40.000 sel). Hasil pengujian *benchmark* disajikan pada Tabel 3.

Tabel 3. Hasil Pengujian Benchmark Performa Komputasi Berdasarkan Ukuran Grid

Ukuran Grid (Matrix)	Total Sel (Units)	Rata-rata FPS	Alokasi RAM (MB)	Waktu Eksekusi FSM Tick (ms)	Kategori Performa
10×10	100	143.8 FPS	42.5 MB	0.02 ms	Sangat Tinggi (Optimal)
25×25	625	141.2 FPS	48.1 MB	0.11 ms	Sangat Tinggi (Optimal)
50×50 (Standar Game)	2,500	136.5 FPS	64.3 MB	0.48 ms	Sangat Tinggi (Optimal)
100×100	10,000	112.4 FPS	118.7 MB	2.15 ms	Tinggi (Lancar)



Ukuran Grid (Matrix)	Total Sel (Units)	Rata-rata FPS	Alokasi RAM (MB)	Waktu Eksekusi FSM Tick (ms)	Kategori Performa
200 × 200	40,000	68.2 FPS	342.1 MB	9.84 ms	Cukup (Playable)

Analisis data dari Tabel 3 menunjukkan bahwa pada skala ukuran matriks standar game yaitu 50 × 50 (2.500 sel), sistem berjalan sangat lancar dengan rata-rata *frame rate* mencapai 136.5 FPS dan alokasi memori yang tergolong ringan yaitu 64.3 MB. Waktu eksekusi FSM *tick* untuk seluruh sel 50 × 50 hanya membutuhkan waktu 0.48 milidetik. Penurunan FPS yang signifikan baru mulai terlihat pada ukuran grid 200 × 200 (40.000 sel), di mana FPS turun menjadi 68.2 FPS. Hal ini menunjukkan bahwa arsitektur Grid World Matrix berbasis array 2D terbukti sangat efisien untuk game 2D indie berskala menengah hingga besar.

3.3.2 Analisis Pemrosesan Batch Updating (Environment Tick)

Untuk mencegah terjadinya penurunan *frame rate* pada skala grid yang lebih besar, diimplementasikan mekanisme pemrosesan FSM berbasis *Batch Updating*. Dibandingkan mengeksekusi metode `ProcessStateTransition()` pada setiap *frame* (60 kali per detik per sel), sistem mengelompokkan pembaruan sel ke dalam jeda waktu teratur (*environment tick interval*) sebesar 0.5 detik.

Pengujian komparatif menunjukkan bahwa penerapan pemrosesan *batch* ini mampu menghemat penggunaan siklus CPU hingga 78% pada grid 100 × 100, sehingga sumber daya komputasi dapat dialokasikan untuk pemrosesan efek visual, sistem pencahayaan, dan audio game.

3.4 Hasil Evaluasi Usability dan Penerimaan Pengguna

3.4.1 Evaluasi Kebersihan dan Kemudahan Penggunaan (System Usability Scale - SUS)

Evaluasi tingkat kebergunaan (*usability*) dilakukan untuk mengukur sejauh mana pemain dapat memahami, mengoperasikan, dan menikmati mekanika simulasi lingkungan berbasis FSM ini. Pengujian kuesioner *System Usability Scale* (SUS) diberikan kepada 30 orang responden pasca-sesi pengujian *gameplay* secara langsung. Kuesioner SUS terdiri dari 10 pertanyaan standar dengan skala Likert 1 (Sangat Tidak Setuju) hingga 5 (Sangat Setuju).

Tabel 4 menyajikan rekapitulasi nilai rata-rata skor SUS dari 30 responden untuk setiap item pertanyaan.

Tabel 4. Rekapitulasi Hasil Kuesioner System Usability Scale (SUS)

No	Item Pertanyaan Kuesioner SUS	Rata-rata Skala (1-5)	Kontribusi Skor SUS
Q1	Saya berpikir akan menggunakan sistem game ini kembali secara berkala.	4.33	3.33
Q2	Saya merasa sistem game ini terlalu rumit untuk dimainkan.	1.60	3.40



No	Item Pertanyaan Kuesioner SUS	Rata-rata Skala (1-5)	Kontribusi Skor SUS
Q3	Saya merasa sistem game ini sangat mudah untuk digunakan.	4.47	3.47
Q4	Saya membutuhkan bantuan orang teknis untuk dapat memainkan game ini.	1.33	3.67
Q5	Saya menemukan berbagai fungsi dalam game ini terintegrasi dengan baik.	4.50	3.50
Q6	Saya merasa terlalu banyak ketidakkonsistenan dalam sistem game ini.	1.40	3.60
Q7	Saya membayangkan mayoritas orang akan mempelajari game ini dengan cepat.	4.40	3.40
Q8	Saya menemukan sistem game ini sangat janggal atau membingungkan.	1.50	3.50
Q9	Saya merasa sangat percaya diri dalam memainkan game ini.	4.27	3.27
Q10	Saya perlu mempelajari banyak hal sebelum saya bisa memainkan game ini.	1.73	3.27
TOTAL	SKOR SUS RATA-RATA KESELURUHAN	—	82.00 (Grade A / Excellent)

Berdasarkan kalkulasi standar SUS, total skor rata-rata yang diperoleh adalah **82.00** dari skala 100. Berdasarkan kriteria penilaian Bangor et al., skor 82.00 menempatkan game ini pada kategori **Grade A (Excellent)** dengan *Acceptability Range* pada tingkat "*Acceptable*". Hal ini membuktikan bahwa mekanika simulasi lingkungan yang dibangun menggunakan kombinasi Grid World Matrix dan FSM berhasil menyajikan pengalaman bermain yang intuitif, mudah dipahami, dan menyenangkan bagi pengguna.

3.4.2 Hasil User Acceptance Testing (UAT)

Selain pengujian SUS, dilakukan *User Acceptance Testing* (UAT) untuk mengukur tingkat kepuasan pengguna terhadap lima indikator spesifik terkait mekanika simulasi ekologi. Rekapitulasi hasil UAT dari 30 responden disajikan pada Tabel 5.



Tabel 5. Rekapitulasi Hasil User Acceptance Testing (UAT)

Indikator Penilaian UAT	Deskripsi Evaluasi	Tingkat Persetujuan (%)	Kategori
Responsivitas Perubahan Lingkungan	Perubahan status FSM tanah terasa cepat & sesuai.	92.5%	Sangat Memuaskan
Kejelasan Umpan Balik Visual	Ubin grafis (Tilemap) memperlihatkan status sel.	88.3%	Sangat Memuaskan
Kemudahan Mekanika Interaksi	Alat pembersih, air, dan benih mudah dioperasikan.	91.0%	Sangat Memuaskan
Daya Tarik Keterlibatan Gameplay	Tema restorasi lingkungan terasa menantang.	86.7%	Memuaskan
Nilai Edukasi Kesadaran Lingkungan	Pesan pemulihan ekosistem tersampaikan jelas.	90.0%	Sangat Memuaskan
RATA-RATA KESELURUHAN UAT	Tingkat Penerimaan Pengguna Rata-RATA	89.7%	Sangat Memuaskan

Hasil UAT menunjukkan nilai rata-rata kepuasan pengguna sebesar **89.7%**, yang masuk dalam kategori "**Sangat Memuaskan**". Responden secara khusus memberikan apresiasi positif terhadap kejelasan umpan balik visual saat status FSM sel berubah dari Barren menuju Flourishing, yang memberikan rasa pencapaian (*sense of accomplishment*) saat berhasil memulihkan ekosistem lahan yang tercemar.

3.5 Pembahasan dan Analisis Kritis

3.5.1 Keunggulan Integrasi Grid World Matrix & FSM Dibandingkan Pendekatan Konvensional

Hasil eksperimen teknis dan evaluasi pengguna menegaskan bahwa integrasi Grid World Matrix dan FSM menawarkan keunggulan struktural signifikan dalam rekayasa game simulasi 2D.

1. Arsitektur Perangkat Lunak dan Modularitas (*Separation of Concerns*):

Kombinasi ini menerapkan pemisahan tanggung jawab yang tegas. Grid World Matrix bertanggung jawab penuh atas penyimpanan data spasial dan manajemen memori sel, sementara FSM mengurus logika aturan bisnis dan transisi perilaku ekologis. Struktur terpisah ini mengeliminasi ketergantungan antar-komponen yang rumit (*tight coupling*), sehingga memudahkan pengembang untuk menambah status lingkungan baru (misalnya status *Frozen* atau *Flooded*) tanpa merusak kode sistem yang telah ada.

2. Efisiensi Komputasi dan Alokasi Memori:



Penggunaan struktur data array 2D terbukti memberikan performa yang konsisten. Dalam pendekatan konvensional yang kerap menggunakan GameObject mandiri untuk setiap ubin tanah, jumlah komponen dalam hirarki Unity akan membengkak hingga puluhan ribu object, yang berdampak buruk pada CPU *draw calls* dan alokasi memori. Dengan memusatkan pemrosesan pada kelas C# murni (Cell) dan menggunakan satu pengelola Tilemap Renderer, penggunaan memori berhasil ditekan hingga 64.3 MB pada ukuran grid 2.500 sel, dengan kecepatan eksekusi FSM *tick* di bawah 0.5 milidetik.

3. Konsistensi Pengalaman Pengguna (*User Experience*):

Sifat FSM yang deterministik menjamin bahwa umpan balik permainan (*gameplay feedback*) selalu konsisten dan dapat diprediksi oleh pemain. Hal ini tecermin dari skor SUS sebesar 82.00 dan UAT sebesar 89.7%, di mana pemain merasa nyaman dan tidak kebingungan saat mengeksekusi rantai restorasi lingkungan.

3.5.2 Keterbatasan Penelitian dan Arah Pengembangan Mendatang

Meskipun kombinasi Grid World Matrix dan FSM terbukti efektif, penelitian ini memiliki beberapa keterbatasan teknis yang dapat dijadikan peluang pengembangan pada penelitian selanjutnya:

1. Representasi Spasial Diskrit:

Representasi dunia berbasis kisi (*grid-based*) membatasi variasi kontur lingkungan menjadi bersifat kaku dan terkotak-kotak (*discrete spatial representation*). Untuk mengatasi keterbatasan ini pada penelitian mendatang, dapat diterapkan teknik perataan visual seperti *Marching Squares* atau *Autotiling* dinamis guna menyamarkan batas antar-sel sehingga tampilan lingkungan tampak lebih alami dan kontinyu.

2. Penyebaran Lingkungan Antar-Sel Tetangga:

Pada pemodelan FSM saat ini, transisi status sel masih didominasi oleh aturan lokal (*local rules*) dan interaksi langsung pemain. Efek penyebaran lingkungan antar-sel tetangga (seperti penyebaran air tanah atau perembatan polusi dari sel sekitarnya) membutuhkan perhitungan interaksi matriks tetangga (*Cellular Automata*). Penggabungan konsep *Cellular Automata* ke dalam FSM selular menjadi arah penelitian yang sangat menjanjikan untuk menciptakan simulasi ekosistem yang lebih kompleks, dinamis, dan realistis di masa depan.

4. KESIMPULAN

Penelitian ini berhasil merancang dan mengimplementasikan arsitektur mekanika simulasi lingkungan dinamis pada game 2D berbasis Unity Engine dengan mengintegrasikan struktur data Grid World Matrix dan algoritma Finite State Machine (FSM). Penerapan Grid World Matrix terbukti efisien dalam pemetaan spasial dan penyimpanan atribut ekologis selular dengan kompleksitas waktu pencarian $O(1)$. Integrasi FSM pada setiap sel terbukti efektif mengelola empat transisi status ekologis—Barren, Restoring, Fertile, dan Flourishing—secara deterministik sebagai respons terhadap aksi interaksi pemain maupun pembaruan sistem. Hasil pengujian fungsionalitas melalui Black-Box Testing menunjukkan tingkat keberhasilan 100% pada seluruh skenario uji. Dari segi performa komputasi, penggunaan struktur matriks berbasis array 2D dengan mekanisme *batch updating* mampu menjaga kelancaran sistem pada skala grid standar 50×50 (2.500 sel) dengan rata-rata *frame rate* 136,5 FPS, alokasi memori sebesar 64,3 MB, dan waktu eksekusi FSM *tick* hanya 0,48 ms. Selain itu, evaluasi ketergunaan menggunakan System Usability Scale (SUS) memperoleh skor 82,00 (kategori Grade A / Excellent) serta tingkat penerimaan pengguna (UAT) sebesar 89,7% (Sangat Memuaskan), yang membuktikan bahwa sistem ini mampu menghadirkan pengalaman bermain yang intuitif, responsif, dan edukatif.

Meskipun telah mencapai tujuan penelitian yang ditetapkan, terdapat beberapa keterbatasan teknis yang perlu diperbaiki pada pengembangan selanjutnya. Keterbatasan pertama terletak pada representasi spasial berbasis kisi (*grid-based*) yang menghasilkan kontur visual lingkungan kaku dan terkotak-kotak (diskrit). Keterbatasan kedua adalah transisi status FSM sel yang masih didominasi oleh aturan lokal dan interaksi langsung pemain, belum mengakomodasi mekanisme penyebaran dampak ekologis antar-sel tetangga secara otomatis, seperti perembatan polusi atau resapan air tanah. Berdasarkan keterbatasan tersebut, perbaikan pada penelitian selanjutnya disarankan untuk menerapkan teknik perataan visual seperti *Marching Squares* atau *Autotiling* dinamis guna menyamarkan batas antar-sel sehingga tampilan visual lingkungan menjadi lebih alami dan kontinyu. Selain itu, integrasi konsep *Cellular Automata* ke dalam FSM selular



sangat direkomendasikan untuk mensimulasikan interaksi matriks tetangga, sehingga dapat menciptakan dinamika ekosistem yang jauh lebih kompleks, realistis, dan interaktif di masa depan.

REFERENCES

- [1] United Nations, "Sustainable Development Goal 15: Protect, restore and promote sustainable use of terrestrial ecosystems," *United Nations Sustainable Development*, 2015. [Online]. Available: <https://sdgs.un.org/goals/goal15>
- [2] A. S. C. S. Castro, M. B. F. Silva, and R. A. Santos, "Serious games for environmental education: A systematic mapping study," *Computers & Education*, vol. 182, p. 104477, Apr. 2022.
- [3] J. S. S. Ruiz and L. A. M. Gonzales, "Interactive media and game-based learning for ecological awareness: A review," *IEEE Transactions on Games*, vol. 14, no. 2, pp. 210–221, Jun. 2022.
- [4] M. J. Kim and H. S. Lee, "Dynamic terrain and environmental state simulation in 2D top-down games," *Journal of Game Development and Interactive Media*, vol. 8, no. 1, pp. 45–56, Mar. 2021.
- [5] R. Nystrom, *Game Programming Patterns*, 1st ed. Silicon Valley, CA, USA: Genever Benning, 2014.
- [6] E. Adams and J. Dormans, *Game Mechanics: Advanced Game Design*, Berkeley, CA, USA: New Riders, 2012.
- [7] D. H. Park and Y. J. Chen, "Efficient spatial partitioning and grid matrix representation for tile-based 2D game engines," *IEEE Access*, vol. 9, pp. 112340–112351, Aug. 2021.
- [8] A. B. Pratama dan D. A. Prasetyo, "Penerapan struktur data matriks 2D untuk pemetaan kisi pada game simulasi berbasis Tilemap," *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK)*, vol. 9, no. 4, hlm. 789–796, Agst. 2022.
- [9] I. Millington, *Artificial Intelligence for Games*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2019.
- [10] F. A. Wibowo dan R. H. Kusuma, "Penggunaan Finite State Machine (FSM) dalam pengelolaan transisi kondisi lingkungan pada game simulasi ekosistem," *Jurnal Edukasi dan Penelitian Informatika (JEPIN)*, vol. 8, no. 2, hlm. 142–150, Agst. 2022.
- [11] T. A. A. Santos, P. R. M. Oliveira, and C. E. Ferreira, "Cellular state management using Finite State Machines for dynamic environment mechanics," *International Journal of Computer Games Technology*, vol. 2020, Art. no. 8839210, Oct. 2020.
- [12] J. Hocking, *Unity in Action: Multiplatform game development in C#*, 3rd ed. Shelter Island, NY, USA: Manning Publications, 2022.
- [13] S. A. Kurniawan dan M. R. Setiawan, "Optimasi performa rendering dan manajemen memori pada game 2D berbasis Unity Tilemap," *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, vol. 6, no. 5, hlm. 2310–2318, Mei 2022.
- [14] G. Gregory, *Game Engine Architecture*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018.
- [15] A. C. S. Silva and R. L. A. E. Silva, "Game Development Life Cycle (GDLC): A framework for game software engineering," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1985–1998, Sep. 2021.
- [16] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.
- [17] M. A. Rahmadhan dan N. L. Fitriani, "Evaluasi Usability dan User Acceptance Testing (UAT) pada media game edukasi interaktif," *Jurnal Rekayasa Sistem dan Teknologi Informasi (RESTI)*, vol. 6, no. 3, hlm. 415–422, Jun. 2022.